

Municipality Management System

AI Architecture Summary

Generated: 2026-06-11

A concise architecture document for a municipality platform using Gemini chatbot services, face recognition attendance, Node.js integration, MongoDB storage, security controls, and operational error handling.

1. Executive Summary

- Use a hybrid AI architecture: Gemini API for citizen-facing chatbot capabilities and municipal-controlled infrastructure for biometric attendance.
- Keep Node.js as the central integration layer. AI components should be internal services behind authenticated backend APIs.
- Use MongoDB for operational data, chatbot records, ticketing, staff attendance, consent, audit logs, and AI error records.
- Design priorities: citizen service quality, biometric consent, least-privilege access,

human escalation, observability, and graceful fallback.

2. AI Requirement Analysis

- Citizen chatbot: multilingual FAQs, service guidance, complaint/request creation, ticket status support, and human handoff.
- Gemini API: called only from the backend through an AI gateway that manages prompts, API keys, rate limits, retries, and policy checks.
- Face recognition attendance: consent-based staff verification with liveness checks, confidence thresholds, encrypted templates, and non-biometric fallback.
- AI governance: audit all model-assisted actions, redact sensitive data, and prevent direct model writes to operational databases.
- Reliability: handle provider outages, low-confidence outputs, camera errors, database failures, and integration timeouts without losing auditability.

3. Chatbot Architecture

- Flow: Citizen Web/Mobile/Kiosk -> Node.js API Gateway -> Auth + Rate Limit + Validation -> Chat Orchestrator -> Guardrails -> Gemini API.
- Knowledge flow: Chat Orchestrator -> Approved Knowledge Base -> MongoDB Vector Search / FAQ Index -> Gemini grounded response.
- Action flow: Chat Orchestrator -> Ticket Service -> MongoDB Tickets -> Human Support Queue when confidence is low or escalation is requested.
- The chatbot should answer using approved municipal content, create tickets through backend APIs, and log all interactions with redacted PII.

4. Face Recognition Attendance Architecture

- Flow: Camera/Mobile Device -> Capture Service -> Device Authentication -> Liveness Detection -> Face Detection -> Embedding -> Match Service.
- Decision flow: Match Service -> Confidence Decision Engine -> Attendance Event for high confidence, Manual Review/PIN fallback for low confidence.
- Enrollment must require explicit staff consent. Store encrypted face templates or embeddings, not raw photos unless legally required.
- Audit every biometric action with staff ID, consent status, device ID, model version, match confidence, timestamp, and reviewer if applicable.

5. Integration Architecture

- Node.js Backend API is the system-of-record interface for citizen channels, staff portals, AI services, tickets, attendance, notifications, and MongoDB.
- AI Gateway centralizes Gemini calls, face service calls, prompt versions, secrets, policy checks, retries, telemetry, and error normalization.
- Face recognition should run as a separate Python FastAPI or compliant vendor-backed service due to specialized compute, privacy, and model lifecycle needs.
- Use asynchronous queues for notifications, audit exports, analytics, and retryable downstream operations.

6. Security and Privacy Design

- Identity: JWT/OAuth2 for users, RBAC for municipal roles, and signed or mTLS service-to-service authentication.
- Secrets: store Gemini API keys and database credentials in a secrets manager with rotation and restricted access.
- AI safety: prompt injection defenses, retrieval allowlists, structured model outputs, tool-call validation, and output filtering.
- Biometrics: explicit consent, non-biometric fallback, encrypted templates, separate access control, retention limits, and revocation workflow.
- Data protection: TLS, encryption at rest, field-level encryption for sensitive fields, PII minimization, and immutable audit trails where required.

7. MongoDB Data Storage Design

- citizens: citizen profile references, contact preferences, and authentication mapping.
- chat_sessions and chat_messages: conversation metadata, redacted transcripts, language, intent, AI metadata, and safety flags.
- knowledge_base and kb_embeddings: approved municipal content and vector-search chunks for retrieval-augmented chatbot answers.
- tickets: complaint/request type, status, department assignment, SLA, citizen reference, and resolution history.
- staff, biometric_consents, face_templates: staff profile, consent lifecycle, encrypted face embeddings, enrollment metadata, and model version.
- attendance_events: timestamp, device ID, match confidence, verification method, and review status.
- ai_audit_logs and ai_errors: request traceability, policy decisions, failures, retry counts, and fallback actions.

8. Error Handling Strategy

- Use centralized Node.js error middleware with a consistent API response: success, code, message, correlationId, and optional details.
- Retry transient Gemini, database, and integration failures using exponential backoff with jitter.
- Use circuit breakers for Gemini and face recognition services to prevent cascading failures.
- Fallbacks: human chatbot escalation, manual attendance, queue-based retries, and safe user-facing messages.
- Never expose stack traces, prompts, embeddings, biometric data, API keys, or internal service details to clients.

9. Technology Recommendations

- Gemini API with the official Google GenAI SDK for chatbot reasoning, summarization, and structured responses.
- Node.js with Express or NestJS for APIs, RBAC middleware, service orchestration, queues, and observability hooks.
- MongoDB Atlas or self-managed MongoDB based on data residency. Use MongoDB Vector Search if Atlas is approved.
- Python FastAPI, OpenCV/MediaPipe, and InsightFace/FaceNet or a compliant managed biometric vendor for face recognition.
- Redis for rate limiting and short-lived context; BullMQ/RabbitMQ/Kafka for asynchronous jobs; Vault or cloud secrets manager for secrets.

10. Final End-to-End Architecture

- Citizen and staff clients connect to the Node.js backend through authenticated, rate-limited APIs.
- The AI Gateway routes chatbot requests to Gemini and biometric requests to the face recognition service.
- MongoDB stores operational records, chatbot metadata, attendance records, consent, templates, audit logs, and AI errors.
- Human review queues handle sensitive chatbot issues, low-confidence biometric matches, failed liveness checks, and exception cases.
- The complete design supports privacy-aware municipal automation with auditability, resilience, and controlled use of AI.

Architecture Flows

- Chatbot: Citizen UI -> Node.js API -> AI Gateway -> Chat Orchestrator -> Gemini + Knowledge Base -> Ticket/Human Escalation -> MongoDB.
- Face Attendance: Camera -> Liveness -> Face Detection -> Embedding -> Match -> Confidence Decision -> Attendance or Manual Fallback -> Audit Log.
- Integration: Clients -> Node.js Backend -> AI Gateway/Core Services -> MongoDB/Queues/Notifications/Observability.

Additional AI Architecture Sections

Detailed Face Recognition Step Diagram

Camera → FaceDetection → LivenessCheck → FaceEmbedding → MatchwithStored Template
→ ConfidenceCheck → Mark Attendance / ManualFallback

Detailed Chatbot Integration Flow

User Query → Node.js API → AI Gateway → Gemini API → Response Validation → Reply to User

Consent Form / Consent Policy

Staff must provide explicit permission before face enrollment. Staff can request deletion of biometric data. A manual attendance option must always be available.

Data Retention Policy

Chatlogs are stored for 6 months. Attendance records are retained according to municipality regulations. Face templates are deleted when employment ends or consent is withdrawn. Raw face images are not stored after enrollment.

AI Risk & Mitigation Table

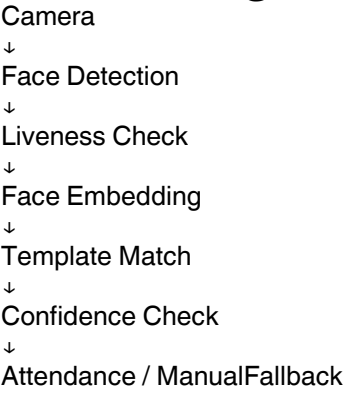
Wrong chatbot answer → Use approved knowledge base. Face mismatch → Manual review/PIN fallback. Gemini failure → Human escalation. Data leakage → Encryption and access control.

Integration Flow Diagram

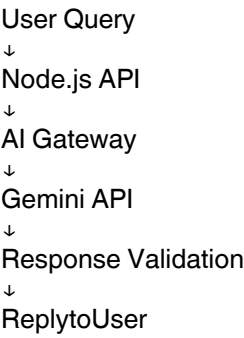
Frontend → Node.js Backend → AI Gateway → Gemini Service + Face Recognition Service → MongoDB Audit Logs & Attendance

Visual Sections for Final Submission

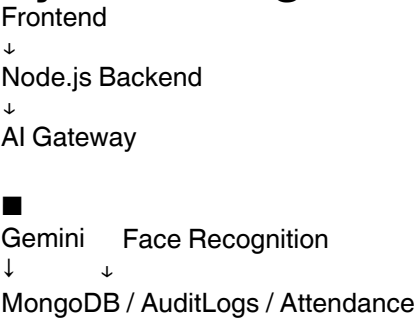
Face Recognition Flowchart



Chatbot Integration Flowchart



System Integration Diagram



AI Risk & Mitigation Matrix

Risk	Mitigation
Wrong chatbot answer	Use approved knowledge base
Face mismatch	Manual review / PIN fallback
Gemini failure	Human escalation

Data leakage	Encryption and access control
--------------	-------------------------------

Consent & Retention Policy

Staff consent is mandatory before biometric enrollment. Staff may request deletion of biometric data. Manual attendance must remain available. Chat logs are retained for 6 months. Face templates are removed when consent is withdrawn or employment ends. Raw face images are not stored after enrollment.